

Guide de Référence

PowerShell & Unity Catalog

sur Azure Databricks

Commandes essentielles · Tableaux structurés · Exemples pratiques

Azure Databricks · Unity Catalog · PowerShell / Databricks CLI

Mai 2026

■ 1. Connexion & Configuration

1.1 Installation des prérequis

■ Ces commandes s'exécutent dans PowerShell 7+ sur Windows, macOS ou Linux.

Commande	Description	Remarque
<code>Install-Module -Name Az -AllowClobber -Force</code>	Installe le module Az (Azure PowerShell)	Nécessite PowerShell 7+
<code>pip install databricks-cli</code>	Installe la CLI Databricks (Python)	Python 3.8+ requis
<code>winget install Databricks.DatabricksCLI</code>	Installe la CLI v2 via WinGet	Windows uniquement
<code>databricks --version</code>	Vérifie la version de la CLI installée	
<code>Get-Module -Name Az -ListAvailable</code>	Liste les versions du module Az disponibles	

1.2 Authentification Azure & Databricks

Commande	Description	Remarque
<code>Connect-AzAccount</code>	Connexion interactive à Azure AD	Ouvre le navigateur
<code>Connect-AzAccount -ServicePrincipal -Tenant \$tenantId -Credential \$cred</code>	Connexion via Service Principal (CI/CD)	Recommandé en prod
<code>databricks configure --token</code>	Configure la CLI avec un Personal Access Token	Stocke dans <code>~/.databrickscfg</code>
<code>databricks configure --aad-token</code>	Configure la CLI avec un token Azure AD	OAuth 2.0 AAD
<code>\$env:DATABRICKS_HOST = 'https://.azuredatabricks.net'</code>	Définit l'URL du workspace en variable d'env	
<code>\$env:DATABRICKS_TOKEN = ''</code>	Définit le PAT en variable d'environnement	
<code>Get-AzAccessToken -ResourceUrl 'https://management.azure.com/'</code>	Récupère un token d'accès Azure	

■■ 2. Gestion du Metastore

■ Le Metastore est le référentiel central de gouvernance de Unity Catalog. Un seul metastore par région Azure.

2.1 Opérations sur le Metastore

Commande	Description	Remarque
<code>databricks unity-catalog metastores list</code>	Liste tous les metastores disponibles	
<code>databricks unity-catalog metastores get --id</code>	Détails d'un metastore spécifique	
<code>databricks unity-catalog metastores create --name --storage-root</code>	Crée un nouveau metastore Unity Catalog	<code>storage-root</code> = ADLS Gen2
<code>databricks unity-catalog metastores delete --id --force</code>	Supprime un metastore (irréversible)	■ Dangereux

Commande	Description	Remarque
<code>databricks unity-catalog metastores assign --metastore-id --workspace-id --default-catalog-name</code>	Assigne un metastore à un workspace	
<code>databricks unity-catalog metastores unassign --workspace-id</code>	Désassigne le metastore du workspace	
<code>databricks unity-catalog metastores update --id --delta-sharing-organization-name</code>	Met à jour les paramètres du metastore	
<code>databricks unity-catalog metastores current</code>	Affiche le metastore actif du workspace courant	

■ 3. Gestion des Catalogues

■ Un catalogue est le premier niveau de la hiérarchie Unity Catalog : *Catalog > Schema > Table*.

3.1 CRUD Catalogues

Commande	Description	Remarque
<code>databricks unity-catalog catalogs list</code>	Liste tous les catalogues du metastore	
<code>databricks unity-catalog catalogs get --name</code>	Affiche les détails d'un catalogue	
<code>databricks unity-catalog catalogs create --name</code>	Crée un nouveau catalogue	
<code>databricks unity-catalog catalogs create --name --storage-root</code>	Crée un catalogue avec stockage externe	<i>ADLS Gen2 requis</i>
<code>databricks unity-catalog catalogs update --name --comment 'description'</code>	Met à jour le commentaire du catalogue	
<code>databricks unity-catalog catalogs delete --name --force</code>	Supprime le catalogue et son contenu	■ <i>Cascade</i>
<code>databricks unity-catalog catalogs list ConvertFrom-Json</code>	Convertit la sortie JSON en objet PS	<i>PowerShell natif</i>

■■ 4. Gestion des Schémas (Bases)

4.1 CRUD Schémas

Commande	Description	Remarque
<code>databricks unity-catalog schemas list --catalog-name</code>	Liste les schémas d'un catalogue	
<code>databricks unity-catalog schemas get --full-name</code>	Détails d'un schéma (notation pointée)	
<code>databricks unity-catalog schemas create --name --catalog-name</code>	Crée un schéma dans le catalogue	
<code>databricks unity-catalog schemas create --name --catalog-name --storage-root</code>	Crée un schéma avec stockage externe	
<code>databricks unity-catalog schemas update --full-name --comment 'desc'</code>	Met à jour la description du schéma	

Commande	Description	Remarque
<code>databricks unity-catalog schemas delete --full-name --force</code>	Supprime le schéma et ses tables	■ <i>Cascade</i>
<code>databricks unity-catalog schemas list --catalog-name ConvertFrom-Json Select-Object name</code>	Extrait uniquement les noms des schémas	

■ 5. Gestion des Tables & Vues

5.1 Exploration des Tables

Commande	Description	Remarque
<code>databricks unity-catalog tables list --catalog-name --schema-name</code>	Liste les tables d'un schéma	
<code>databricks unity-catalog tables get --full-name</code>	Détails complets d'une table	
<code>databricks unity-catalog tables exists --full-name</code>	Vérifie l'existence d'une table	
<code>databricks unity-catalog tables list-summaries --catalog-name</code>	Résumé de toutes les tables du catalogue	
<code>databricks unity-catalog tables delete --full-name</code>	Supprime une table Unity Catalog	■ <i>Irréversible</i>
<code>databricks unity-catalog tables update --full-name --comment 'desc'</code>	Ajoute/met à jour un commentaire de table	

5.2 Colonnes & Métadonnées

Commande	Description	Remarque
<code>databricks unity-catalog tables get --full-name ConvertFrom-Json Select-Object -ExpandProperty columns</code>	Affiche les colonnes et leurs types	
<code>databricks unity-catalog tables get --full-name ConvertFrom-Json Select-Object -ExpandProperty properties</code>	Propriétés personnalisées de la table	
<code>databricks unity-catalog tables list --catalog-name --schema-name ConvertFrom-Json Where-Object {\$_table_type -eq 'EXTERNAL'}</code>	Filtre les tables externes uniquement	
<code>databricks unity-catalog tables list --catalog-name --schema-name ConvertFrom-Json Where-Object {\$_table_type -eq 'MANAGED'}</code>	Filtre les tables managées uniquement	

■ 6. Gestion des Permissions (RBAC)

■ Unity Catalog utilise un modèle RBAC hiérarchique : les permissions se propagent du metastore vers les tables.

6.1 Consulter les Permissions

Commande	Description	Remarque
<code>databricks unity-catalog permissions get --securable-type catalog --name</code>	Permissions sur un catalogue	
<code>databricks unity-catalog permissions get --securable-type schema --name</code>	Permissions sur un schéma	
<code>databricks unity-catalog permissions get --securable-type table --name</code>	Permissions sur une table	
<code>databricks unity-catalog permissions get --securable-type metastore --name</code>	Permissions au niveau metastore	<i>Admin requis</i>
<code>databricks unity-catalog effective-permissions get --securable-type table --name</code>	Permissions effectives (héritées incluses)	<i>Vue consolidée</i>
<code>databricks unity-catalog permissions get --securable-type storage-credential --name</code>	Permissions sur une storage credential	

6.2 Accorder / Révoquer des Permissions

Commande	Description	Remarque
<code>databricks unity-catalog permissions update --securable-type catalog --name --json '{"changes":[{"principal":"user@corp.com","add":["USE_CATALOG"]}]}'</code>	Accorde USE_CATALOG à un utilisateur	
<code>databricks unity-catalog permissions update --securable-type schema --name --json '{"changes":[{"principal":"grp","add":["USE_SCHEMA","SELECT"]}]}'</code>	Accorde USE_SCHEMA + SELECT à un groupe	
<code>databricks unity-catalog permissions update --securable-type table --name --json '{"changes":[{"principal":"user@corp.com","remove":["SELECT"]}]}'</code>	Révoque SELECT sur une table	
<code>databricks unity-catalog permissions update --securable-type catalog --name --json '{"changes":[{"principal":"account users","add":["USE_CATALOG"]}]}'</code>	Accorde l'accès à tous les utilisateurs du compte	<i>Groupe système</i>

6.3 Privilèges disponibles par type d'objet

Objet	Privilèges	Portée
METASTORE	CREATE_CATALOG, CREATE_EXTERNAL_LOCATION, CREATE_STORAGE_CREDENTIAL, MANAGE_ALLOWLIST	<i>Niveau global</i>
CATALOG	USE_CATALOG, CREATE_SCHEMA, CREATE_TABLE, CREATE_VIEW, CREATE_FUNCTION	<i>Niveau catalogue</i>
SCHEMA	USE_SCHEMA, CREATE_TABLE, CREATE_VIEW, CREATE_FUNCTION, MODIFY, SELECT	<i>Niveau schéma</i>
TABLE / VIEW	SELECT, MODIFY, ALL_PRIVILEGES	<i>Niveau table</i>

Objet	Privilèges	Portée
EXTERNAL LOCATION	CREATE_EXTERNAL_TABLE, READ_FILES, WRITE_FILES	Accès stockage
STORAGE CREDENTIAL	CREATE_EXTERNAL_LOCATION, READ_FILES, WRITE_FILES	Crédentiels

■ 7. Utilisateurs, Groupes & Service Principals

7.1 Gestion des Utilisateurs

Commande	Description	Remarque
<code>databricks accounts users list</code>	Liste tous les utilisateurs du compte	Niveau compte
<code>databricks accounts users get --user-id</code>	Détails d'un utilisateur	
<code>databricks accounts users create --json '{"user Name":"user@corp.com", "displayName":"Nom"}'</code>	Crée un utilisateur dans le compte	
<code>databricks accounts users delete --user-id</code>	Supprime un utilisateur du compte	■ Irréversible
<code>databricks workspace users list</code>	Liste les utilisateurs du workspace actif	Niveau workspace
<code>databricks workspace users get --user-id</code>	Infos d'un utilisateur workspace	

7.2 Gestion des Groupes

Commande	Description	Remarque
<code>databricks accounts groups list</code>	Liste tous les groupes du compte	
<code>databricks accounts groups get --group-id</code>	Détails d'un groupe	
<code>databricks accounts groups create --json '{"displayName":"data-engineers"}'</code>	Crée un groupe dans le compte	
<code>databricks accounts groups patch --group-id --json '{"Operations":[{"op":"add","path":"members","value":[{"value":""}]}]}'</code>	Ajoute un membre au groupe	SCIM API
<code>databricks accounts groups patch --group-id --json '{"Operations":[{"op":"remove","path":"members","value":[{"value":""}]}]}'</code>	Retire un membre du groupe	
<code>databricks accounts groups delete --group-id</code>	Supprime le groupe	■
<code>databricks workspace groups list</code>	Groupes au niveau workspace uniquement	

7.3 Service Principals

Commande	Description	Remarque
<code>databricks accounts service-principals list</code>	Liste les service principals du compte	
<code>databricks accounts service-principals create --json '{"displayName":"my-sp"}'</code>	Crée un service principal	
<code>databricks accounts service-principals get --service-principal-id</code>	Détails d'un service principal	

Commande	Description	Remarque
<code>databricks accounts service-principals delete --service-principal-id</code>	Supprime le service principal	
<code>databricks workspace service-principals list</code>	Service principals du workspace courant	

■ ■ 8. External Locations & Storage Credentials

■ Les External Locations pointent vers des conteneurs ADLS Gen2. Elles s'appuient sur des Storage Credentials (Managed Identity ou Service Principal).

8.1 Storage Credentials

Commande	Description	Remarque
<code>databricks unity-catalog storage-credentials list</code>	Liste toutes les storage credentials	
<code>databricks unity-catalog storage-credentials get --name</code>	Détails d'une storage credential	
<code>databricks unity-catalog storage-credentials create --json @cred.json</code>	Crée une credential depuis un fichier JSON	<i>Voir format ci-dessous</i>
<code>databricks unity-catalog storage-credentials validate --name</code>	Valide la credential (teste l'accès)	<i>Recommandé après création</i>
<code>databricks unity-catalog storage-credentials delete --name</code>	Supprime une storage credential	

■ Exemple de fichier cred.json (Managed Identity) :

```
{"name":"mi-cred","azure_managed_identity":{"access_connector_id":"/subscriptions/.../accessConnectors/my-connector"}}
```

8.2 External Locations

Commande	Description	Remarque
<code>databricks unity-catalog external-locations list</code>	Liste toutes les external locations	
<code>databricks unity-catalog external-locations get --name</code>	Détails d'une external location	
<code>databricks unity-catalog external-locations create --name --url abfss://@.dfs.core.windows.net/ --storage-credential-name</code>	Crée une external location ADLS Gen2	
<code>databricks unity-catalog external-locations validate --name</code>	Valide l'accès à l'external location	
<code>databricks unity-catalog external-locations update --name --comment 'desc'</code>	Met à jour la description	
<code>databricks unity-catalog external-locations delete --name --force</code>	Supprime l'external location	

■ ■ 9. Lineage, Tags & Data Classification

9.1 Tags sur les objets Unity Catalog

Commande	Description	Remarque
<code>databricks unity-catalog tables update --full-name --properties '{"env":"prod","owner":"team-data"}'</code>	Ajoute des propriétés/tags à une table	
<code>databricks unity-catalog schemas update --full-name --properties '{"domain":"finance"}'</code>	Ajoute des tags à un schéma	

Commande	Description	Remarque
<code>databricks unity-catalog catalogs update --name --properties '{"classification":"confidential"}'</code>	Classe un catalogue comme confidentiel	
<code>databricks unity-catalog tables get --full-name ConvertFrom-Json Select-Object -ExpandProperty properties</code>	Lit les tags/propriétés d'une table	
<code>databricks unity-catalog tables list --catalog-name --schema-name ConvertFrom-Json Where-Object {\$_.properties.env -eq 'prod'}</code>	Filtre les tables par tag 'env=prod'	<i>PowerShell</i>

9.2 Audit & Monitoring avec PowerShell

Commande	Description	Remarque
<code>Invoke-RestMethod -Uri "\$env:DATABRICKS_HOST/api/2.0/unity-catalog/audit-logs" -Headers @{Authorization="Bearer \$env:DATABRICKS_TOKEN"}</code>	Interroge les logs d'audit via l'API REST	
<code>Get-AzDatabricksWorkspace -ResourceGroupName</code>	Infos du workspace via module Az	<i>Module Az.Databricks</i>
<code>Get-AzMonitorDiagnosticSetting -ResourceId</code>	Paramètres de diagnostic Azure Monitor	
<code>Set-AzDiagnosticSetting -Name 'unity-audit' -ResourceId -WorkspaceId -Enabled \$true</code>	Active les logs vers Log Analytics Workspace	
<code>Search-AzGraph -Query "Resources where type == 'microsoft.databricks/workspaces'"</code>	Recherche les workspaces Databricks via ARG	<i>Module Az.ResourceGraph</i>

■ 10. Scripts PowerShell Utilitaires

10.1 Export complet des permissions d'un catalogue

```
# Export des permissions - Unity Catalog $catalog = "mon_catalogue" $output = @() # Permissions
du catalogue $catPerms = databricks unity-catalog permissions get ` --securable-type catalog
--name $catalog | ConvertFrom-Json $output += $catPerms.privilege_assignments | Select-Object
@{N='Objet';E={"CATALOG:$catalog"}}, principal, privileges # Permissions de chaque schéma
$schemas = databricks unity-catalog schemas list ` --catalog-name $catalog | ConvertFrom-Json
foreach ($sch in $schemas) { $p = databricks unity-catalog permissions get ` --securable-type
schema --name "$catalog.$($sch.name)" | ConvertFrom-Json $output += $p.privilege_assignments |
Select-Object @{N='Objet';E={"SCHEMA:$catalog.$($sch.name)}}, principal, privileges } $output |
Export-Csv -Path "permissions_$catalog.csv" -NoTypeInfoation Write-Host "Export terminé :
permissions_$catalog.csv" -ForegroundColor Green
```

10.2 Inventaire complet des tables (toutes les bases)

```
# Inventaire de toutes les tables Unity Catalog $metastore = databricks unity-catalog metastores
current | ConvertFrom-Json Write-Host "Metastore : $($metastore.name)" -ForegroundColor Cyan
$inventory = @() $catalogs = databricks unity-catalog catalogs list | ConvertFrom-Json foreach
($cat in $catalogs) { $schemas = databricks unity-catalog schemas list ` --catalog-name $cat.name
| ConvertFrom-Json foreach ($sch in $schemas) { $tables = databricks unity-catalog tables list `
--catalog-name $cat.name --schema-name $sch.name | ConvertFrom-Json foreach ($tbl in $tables) {
$inventory += [PSCustomObject]@{ Catalogue = $cat.name Schema = $sch.name Table = $tbl.name Type
= $tbl.table_type DataFormat = $tbl.data_source_format FullName = $tbl.full_name } } } }
$inventory | Export-Csv "inventory_unity_catalog.csv" -NoTypeInfoation Write-Host "Total :
 $($inventory.Count) tables" -ForegroundColor Green
```

10.3 Attribution en masse de permissions

```
# Attribution en masse : SELECT sur toutes les tables d'un schéma param( [string]$Catalog =
"mon_catalogue", [string]$Schema = "mon_schema", [string]$Group = "data-analysts" ) $tables =
databricks unity-catalog tables list ` --catalog-name $Catalog --schema-name $Schema |
ConvertFrom-Json foreach ($tbl in $tables) { $body =
@{changes=@(@{principal=$Group;add=@("SELECT")})} | ConvertTo-Json -Depth 5 databricks
unity-catalog permissions update ` --securable-type table --name $tbl.full_name --json $body
Write-Host " SELECT accordé sur $($tbl.full_name)" -ForegroundColor Yellow } Write-Host "Terminé
: $($tables.Count) tables mises à jour." -ForegroundColor Green
```

■ 11. Référence Rapide – Aide-mémoire

Action	Syntaxe générique	Notes
Lister les objets	... list [--catalog-name --schema-name]	<i>catalogs / schemas / tables</i>
Créer	... create --name [--json @file.json]	<i>Toujours vérifier les prérequis</i>
Lire les détails	... get --name --full-name	<i>full-name pour schémas et tables</i>
Modifier	... update --name --comment 'desc'	
Supprimer	... delete --name [--force]	<i>--force ignore les objets enfants</i>
Lire les permissions	permissions get --securable-type	<i>catalog / schema / table</i>
Modifier les permissions	permissions update --json '{changes:[...]}'	<i>add / remove par principal</i>

Action	Syntaxe générique	Notes
Valider le stockage	... validate --name	<i>storage-credentials et external-locations</i>
Convertir JSON → PS	... ConvertFrom-Json	<i>Chaîne native PowerShell</i>
Filtrer les résultats	... ConvertFrom-Json Where-Object {...}	<i>Tout filtre PowerShell valide</i>
Exporter en CSV	... Export-Csv -Path fichier.csv	<i>-NoTypeInfoInformation conseillé</i>
Aide CLI	databricks unity-catalog --help	<i>Aide contextuelle en ligne</i>

Ressources officielles : docs.databricks.com · learn.microsoft.com/azure/databricks · github.com/databricks/databricks-sdk-py

Ce guide couvre la CLI Databricks v0.200+ et le module Az PowerShell. Vérifiez la compatibilité des versions.